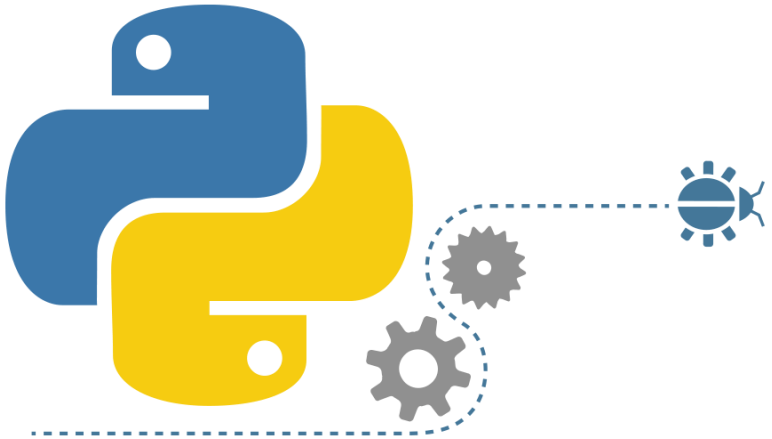
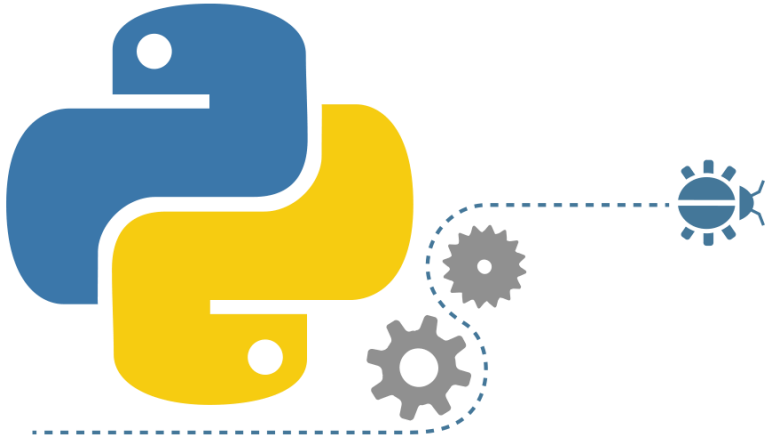
Làm việc với File và Thư mục

1. File là gì?
2. Mở File trong Python
3. Đóng File trong Python
4. Ghi File trong Python
5. Đọc File trong Python
6. Một số phương thức làm việc với File
7. Hiển thị thư mục hiện tại
8. Thay đổi thư mục hiện tại
9. Danh sách thư mục và file
10. Tạo một thư mục mới
11. Đổi tên thư mục hoặc tên file
12. Xóa bỏ thư mục hoặc file
13. Các bài tập rèn luyện



1. File là gì?



1. File là gì?

File hay còn gọi là tệp, tập tin. File là tập hợp của các thông tin được đặt tên và lưu trữ trên bộ nhớ máy tính như đĩa cứng, đĩa mềm, CD, DVD,...

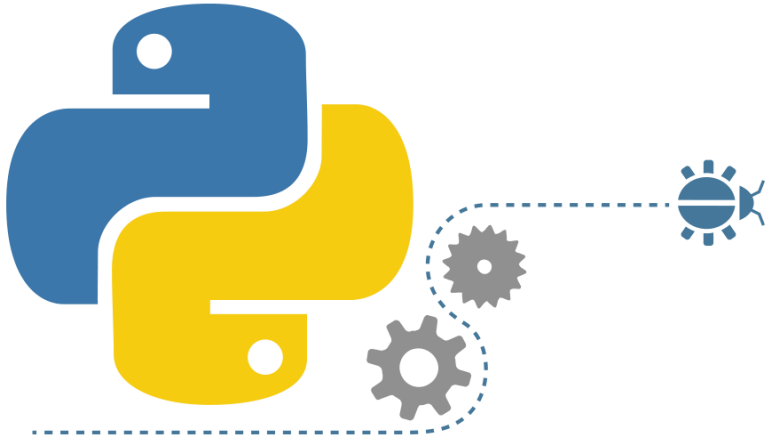
Khi muốn đọc hoặc ghi file, chúng ta cần phải mở file trước. Khi hoàn thành, file cần phải được đóng lại để các tài nguyên được gắn với file được giải phóng.

Do đó, trong Python, một thao tác với file diễn ra theo thứ tự sau.

1. Mở tệp tin
2. Đọc hoặc ghi

Đóng tệp

2. Mở File trong Python



2. Mở File trong Python

open()

Hàm này trả về đối tượng file hay còn gọi là “handle” vì bạn có thể thực hiện các hoạt động đọc, ghi, sửa đổi trên file đó.

```
>>> f = open("test.txt") # mở file cùng thư mục với file hiện tại
>>> f = open("C:/Python33/README.txt") # mở file ở thư mục khác, đường dẫn đầy đủ
```

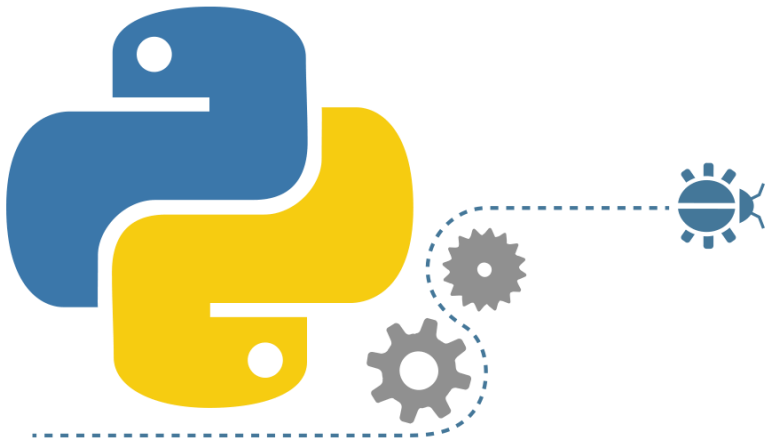
Xác định tập tin được mở ra để làm gì như read, write, append,... đây là thông số tùy chọn có thể có hoặc không

```
f = open("test.txt") # mở file mode 'r' hoăc 'rt' để đọc
f = open("test.txt",'w') # mở file mode ‘w’ để ghi
f = open("img.bmp",'r+b') # mở file mode ‘r+b’ để đọc và ghi dạng nhị phân
```

Khi làm việc với các tệp ở chế độ văn bản, bạn nên chỉ định loại mã hóa.

```
f = open("test.txt",mode = 'r',encoding = 'utf-8')
```

3. Đóng File trong Python



3. Đóng File trong Python

close()

Đóng file để đảm bảo quy chế đóng mở và giải phóng bộ nhớ cho chương trình nên điều này là cần thiết

```
f = open("test.txt",encoding = 'utf-8')
# thực hiện các thao tác với tệp
f.close()
```

Để đảm bảo hơn, bạn nên sử dụng khối lệnh try...finally

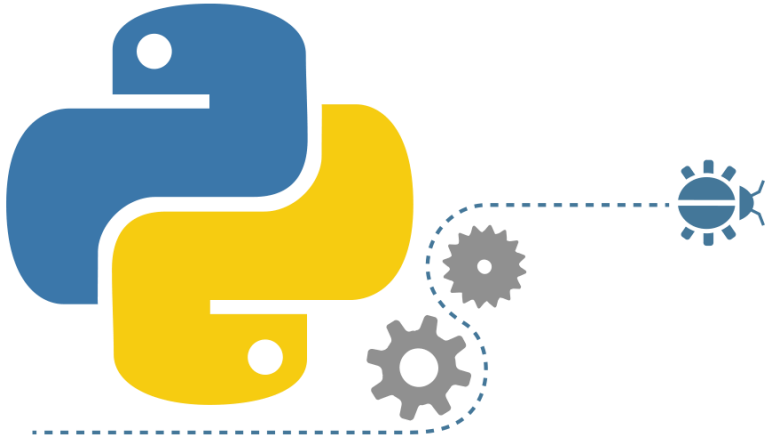
```
try:
 f = open("test.txt",encoding = 'utf-8')
 # thực hiện các thao tác với tệp
finally:
 f.close()
```

Lệnh *with* cho chúng ta bảo đảm rằng file luôn luôn được đóng mà không cần biết những logic xử lý bên trong

```
with open("test.txt",encoding = 'utf-8') as f:
 # thực hiện các thao tác với tệp
```

sử dụng *with* cho chúng ta cách viết code ngắn gọn hơn hẳn

4. Ghi File trong Python



4. Ghi File trong Python

Để ghi một file ta cần mở file bằng cú pháp để ghi, sử dụng mode write ‘w’, append ‘a’ hoặc mode tạo độc quyền ‘x’

Cẩn thận với chế độ ‘w’, vì nó ghi đè lên nội dung nếu file đã tồn tại, các dữ liệu trước đó sẽ bị xóa.

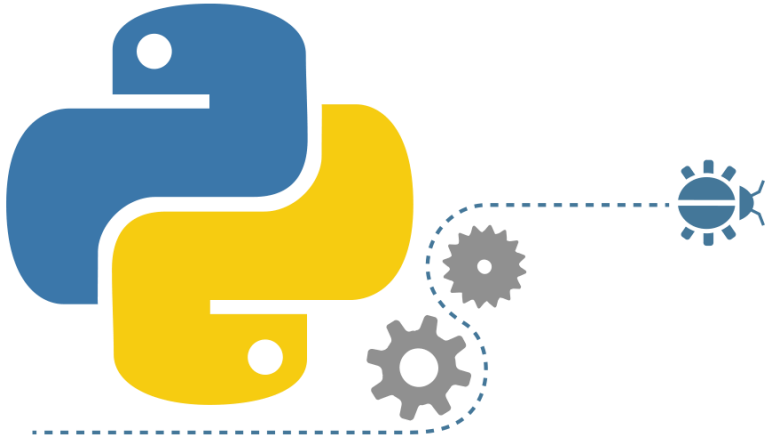
Nếu bạn ghi vào file dạng nhị phân các chuỗi văn bản hoặc chuỗi dạng byte thì kết quả trả về sẽ là số kí tự được ghi vào file

```
with open("test.txt",'w',encoding = 'utf-8') as f:
 f.write("I love coding\n")
 f.write("Kiến thức - Kinh nghiệm - Hỏi đáp\n\n")
 f.write("Ilovecoding.com\n")
```

```
I love coding
Kiến thức - Kinh nghiệm - Hỏi đáp

Ilovecoding.com
```

5. Đọc File trong Python



5. Đọc File trong Python

read(size)

để lấy về dữ liệu có kích thước bằng size

```
f = open("test.txt",'r',encoding = 'utf-8')
a = f.read(13) # đọc 13 kí tự đầu tiên
print('Nội dung 11 kí tự đầu là:\n', (a))
b = f.read(35) # đọc 35 kí tự tiếp theo
print('Nội dung 35 kí tự tiếp theo là:\n', (b))
c = f.read() # đọc phần còn lại
print('Nội dung phần còn lại là:\n', (c))
```

```
Nội dung 13 kí tự đầu là:
 I love coding

Nội dung 35 kí tự tiếp theo là:
 Kiến thức - Kinh nghiệm - Hỏi đáp

Nội dung phần còn lại là:
 Ilovecoding.com
```

tell() và seek()

tell() cho bạn biết vị trí hiện tại bên trong file, việc đọc và ghi tiếp theo sẽ diễn ra tiếp tục trên các byte đó.

seek() thay đổi vị trí hiện tại bên trong file.

```
 f = open("test.txt",'r',encoding = 'utf-8')
a = f.read(13) # đọc 13 kí tự đầu tiên
print('Nội dung là: \n', (a))
b = f.tell() # Kiểm tra vị trí hiện tại
print ('Vị trí hiện tại: ', (b))
f.seek(0) # Đặt lại vị trí con trỏ tại vị trí đầu file
c = f.read()
print('Nội dung mới là: \n', (c))
```

```
Nội dung là:
I love coding

Vị trí hiện tại: 14
Nội dung mới là:
I love coding
Kiến thức - Kinh nghiệm - Hỏi đáp

Ilovecoding.com
```

5. Đọc File trong Python

readline()

cho phép đọc từng dòng trong file

```
f = open("test.txt",'r',encoding = 'utf-8')
a = f.readline()
print ('Nội dung dòng đầu: ', (a))
b = f.readline()
print ('Nội dung dòng 2: ', (b))
c = f.readline()
print ('Nội dung dòng 3: ', (c))
d = f.readline()
print ('Nội dung dòng 4: ', (d))
```

```
Nội dung dòng đầu: I love coding

Nội dung dòng 2: Kiến thức - Kinh nghiệm - Hỏi đáp

Nội dung dòng 3:

Nội dung dòng 4: Ilovecoding.com
```

readlines()

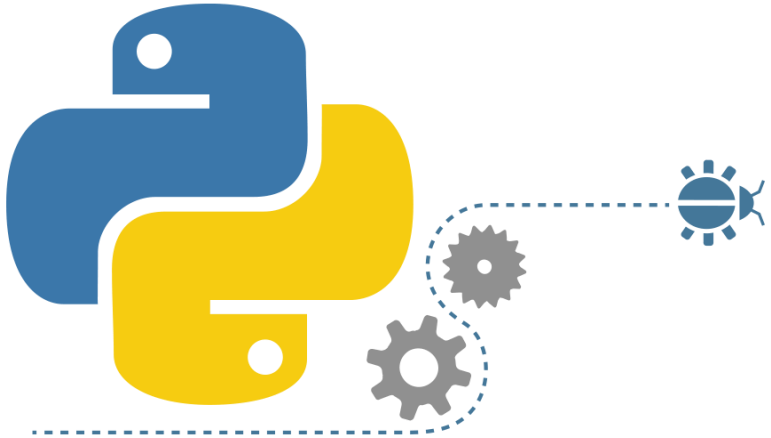
trả về toàn bộ các dòng còn lại trong file và trả về giá trị rỗng khi kết thúc file

```
f = open("test.txt",'r',encoding = 'utf-8')
a = f.readline()
print ('Nội dung dòng đầu: ', (a))
b = f.readlines()
print ('Nội dung các dòng còn lại: \n', (b))
c = f.readlines()
print ('Nội dung các dòng còn lại: \n', (c))
```

```
Nội dung dòng đầu: I love coding

Nội dung các dòng còn lại:
 ['Kiến thức - Kinh nghiệm - Hỏi đáp\n', '\n', 'Ilovecoding.com\n']
Nội dung các dòng còn lại:
 []
```

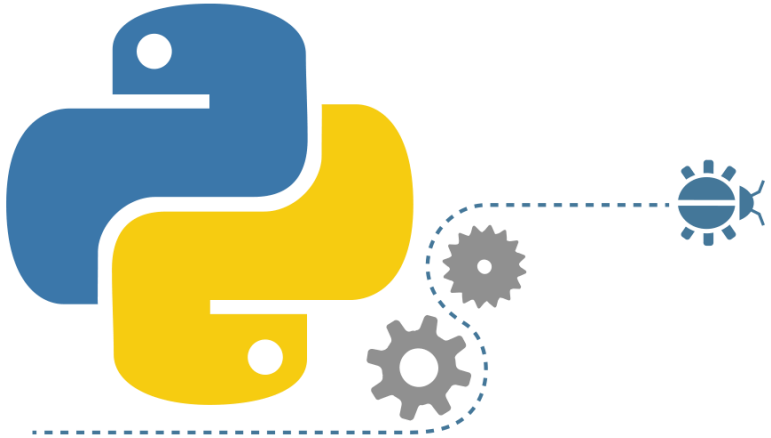
6. Một số phương thức làm việc với File



6. Một số phương thức làm việc với File

PHƯƠNG THỨC	MÔ TẢ
close()	Đóng một file đang mở. Nó không thực thi được nếu tập tin đã bị đóng.
fileno()	Trả về một số nguyên mô tả file (file descriptor).
flush()	Xóa sạch bộ nhớ đệm của luồng file.
isatty()	Trả về TRUE nếu file được kết nối với một thiết bị đầu cuối.
read(n)	Đọc n kí tự trong file.
readable()	Trả về TRUE nếu file có thể đọc được.
readline(n=-1)	Đọc và trả về một dòng từ file. Đọc nhiều nhất n byte/ký tự nếu được chỉ định.
readlines(n=-1)	Đọc và trả về một danh sách các dòng từ file. Đọc nhiều nhất n byte/ký tự nếu được chỉ định.
seek(offset,from=SEEK_SET)	Thay đổi vị trí hiện tại bên trong file.
seekable()	Trả về TRUE nếu luồng hỗ trợ truy cập ngẫu nhiên.
tell()	Trả về vị trí hiện tại bên trong file.
truncate(size=None)	Cắt gọn kích cỡ file thành kích cỡ tham số size.
writable()	Trả về TRUE nếu file có thể ghi được.
write(s)	Ghi s kí tự vào trong file và trả về.
writelines(lines)	Ghi một danh sách các dòng và file.

7. Hiển thị thư mục hiện tại



7. Hiển thị thư mục hiện tại

getcwd() hiển thị thư mục đang làm việc hiện tại, trả về kết quả dưới dạng một chuỗi

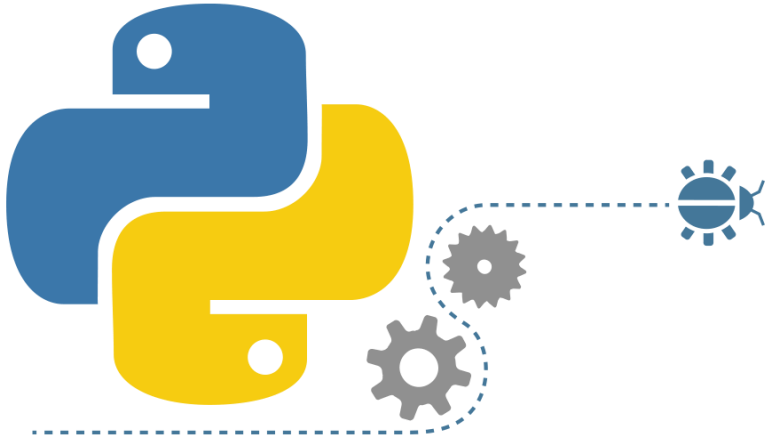
getcwdb() để nhận về kết quả dưới dạng byte

```
>>> import os

>>> os.getcwd()
'C:\\Program Files\\PyScripter'

>>> os.getcwdb()
b'C:\\Program Files\\PyScripter'
```

8. Thay đổi thư mục hiện tại



8. Thay đổi thư mục hiện tại

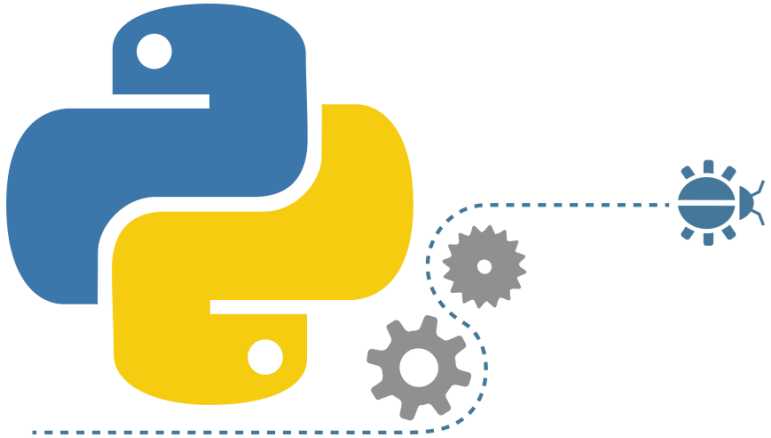
chdir() nhận một tham số là tên thư mục bạn muốn tới từ thư mục hiện tại

Có thể sử dụng cả dấu gạch chéo (/) hoặc dấu gạch chéo ngược (\) để tách các phần tử trong đường dẫn, nhưng tốt nhất vẫn nên sử dụng dấu gạch ngược (\)

```
>>> os.chdir('C:\\Python33')

>>> print(os.getcwd())
C:\Python33
```

9. Danh sách thư mục và file



9. Danh sách thư mục và file

listdir() liệt kê tất cả các tệp và thư mục con bên trong một thư mục

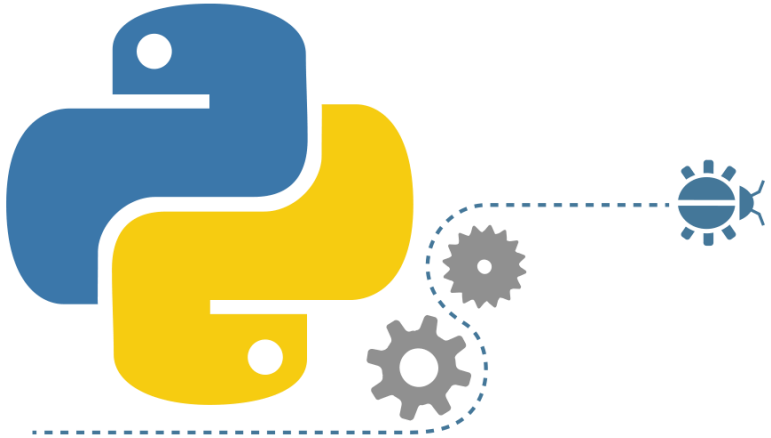
Phương thức này nhận một đường dẫn và trả về danh sách thư mục con và các file trong đường dẫn đó.

Nếu không có đường dẫn nào được chỉ định, kết quả trả về sẽ truy xuất từ thư mục làm việc hiện tại

```
>>> os.listdir()
['DLLs',
'Doc',
'include',
'Lib',
'libs',
'LICENSE.txt',
'NEWS.txt',
'python.exe',
'pythonw.exe',
'README.txt',
'Scripts',
'tcl',
'Tools']
```

```
>>> os.listdir('G:\\')
['$RECYCLE.BIN',
'Movies',
'Music',
'Photos',
'Series',
'System Volume Information']
```

10. Tạo một thư mục mới



10. Tạo một thư mục mới

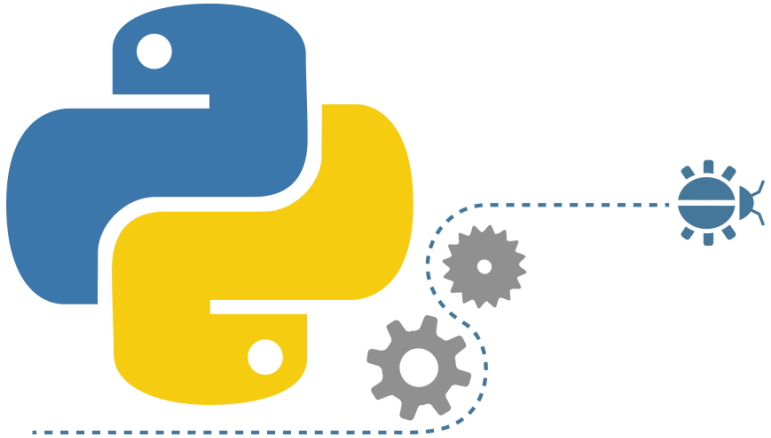
mkdir()

Chọn nơi chứa thư mục mới bằng cách ghi đầy đủ đường dẫn tới nơi muốn tạo
Nếu đường dẫn đầy đủ không được chỉ định, thư mục mới sẽ được tạo trong thư mục làm việc hiện tại

```
>>> os.mkdir('test')

>>> os.listdir()
['test']
```

11. Đổi tên thư mục hoặc tên file



11.Đổi tên thư mục hoặc tên file

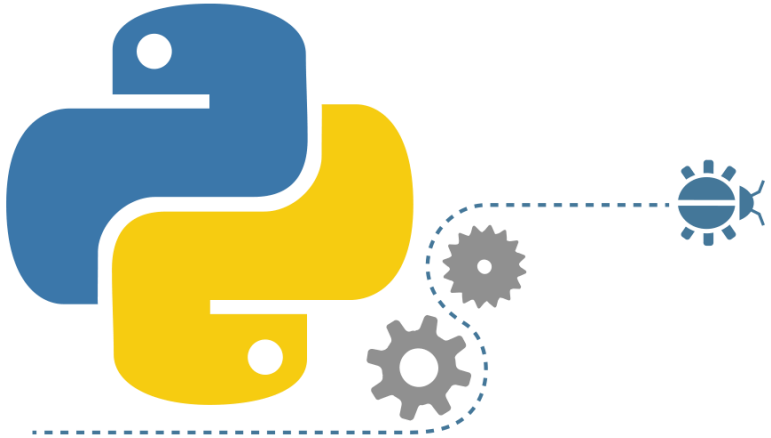
rename() đổi tên một thư mục hoặc một tập tin

```
>>> os.listdir()
['test']

>>> os.rename('test','new_one')

>>> os.listdir()
['new_one']
```

12. Xóa bỏ thư mục hoặc file



12. Xóa bỏ thư mục hoặc file

remove() để gỡ bỏ một file

rmdir() xóa toàn bộ thư mục

```
>>> os.listdir()
['new_one', 'old.txt']

>>> os.remove('old.txt')
>>> os.listdir()
['new_one']

>>> os.rmdir('new_one')
>>> os.listdir()
[]
```

Lưu ý rằng phương thức ***rmdir()*** chỉ có thể xóa các thư mục rỗng.
Để loại bỏ một thư mục không rỗng dùng ***rmtree()*** bên trong module *shutil.*

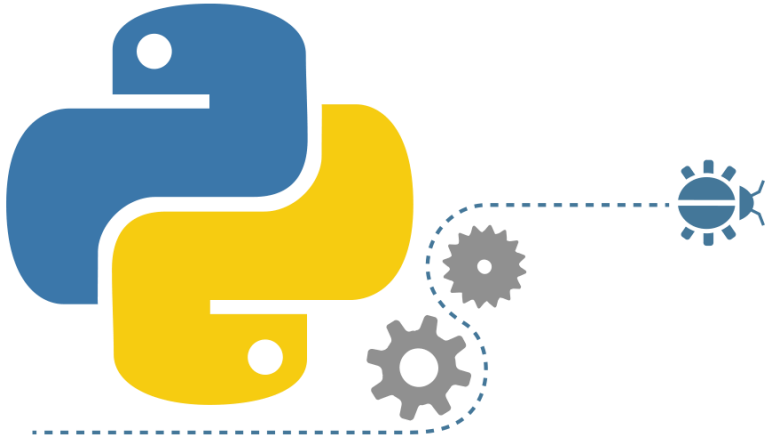
```
>>> os.listdir()
['test']

>>> os.rmdir('test')
Traceback (most recent call last):
...
OSError: [WinError 145] The directory is not empty: 'test'

>>> import shutil

>>> shutil.rmtree('test')
>>> os.listdir()
[]
```

13. Bài tập rèn luyện



Dữ liệu Sản Phẩm

Viết chương trình nhập vào thông tin của một sản phẩm:

Mã: Chuỗi

Tên: Chuỗi

Đơn Giá: Số

Mỗi một Sản phẩm sau khi nhập thành công sẽ lưu nối đuôi vào File theo quy tắc:

MSSP;Tên Sản phẩm; Đơn giá

1) xuất danh sách sản phẩm từ File
2) Sắp xếp Sản phẩm theo đơn giá giảm dần

Dữ liệu Chuỗi số

Cho một tập tin có dữ liệu trên mỗi dòng như dưới đây:

5,6,8,9,-5
-9,5,4,7,8
6,7,8,3,6,46,7,2,-6,-7

1) Viết hàm đọc file, mỗi dòng khởi tạo thành 1 list và xuất ra màn hình
2) Xuất các số âm trên mỗi dòng ra màn hình

Bài tập tự rèn luyện Xử lý File

Câu 1: Trình bày kỹ thuật lưu file và đọc file trong Python

Câu 2: Viết phần mềm Quản Lý sản phẩm

- Mỗi danh mục có: Mã , tên; Một danh mục có nhiều sản phẩm
- Mỗi sản phẩm có: Mã, tên, đơn giá; Mỗi một sản phẩm thuộc về một danh mục.
- Cho phép: lưu mới, sửa, xóa, tìm kiếm, sắp xếp, lưu và đọc Text File

Câu 3: Viết phần mềm quản lý Sinh Viên

- Mỗi một lớp có: Mã lớp, tên; một lớp có nhiều Sinh viên
- Mỗi sinh viên có: mã, tên, năm sinh; Mỗi một sinh viên thuộc về một lớp.
- Cho phép: lưu mới, sửa, xóa, tìm kiếm, sắp xếp, lưu và đọc File